



计算机网络（第7版）

谢希仁 编著



电子工业出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
<http://www.phei.com.cn>

第5章 运输层



5.7

TCP 的流量控制

5.8

TCP 的拥塞控制

5.9

TCP 的运输连接管理



5.7 TCP 的流量 控制

5.7.1

利用滑动窗口实现流量控制

5.7.2

TCP 的传输效率



5.7.1 利用滑动窗口实现流量控制

- 一般说来，我们总是希望数据传输得更快一些。但如果发送方把数据发送得过快，接收方就可能来不及接收，这就会造成数据的丢失。
- **流量控制** (flow control) 就是让发送方的发送速率不要太快，既要让接收方来得及接收，也不要使网络发生拥塞。
- 利用**滑动窗口机制**可以很方便地在 TCP 连接上实现流量控制。



利用可变窗口进行流量控制举例

A 向 B 发送数据。在连接建立时，B 告诉 A：
“我的接收窗口 $rwnd = 400$ (字节)”。



A 发送了序号 1 至 100，还能发送 300 字节

A 发送了序号 101 至 200，还能发送 200 字节

允许 A 发送序号 201 至 500 共 300 字节

A 发送了序号 301 至 400，还能再发送 100 字节新数据

A 发送了序号 401 至 500，不能再发送新数据了

A 超时重传旧的数据，但不能发送新的数据

允许 A 发送序号 501 至 600 共 100 字节

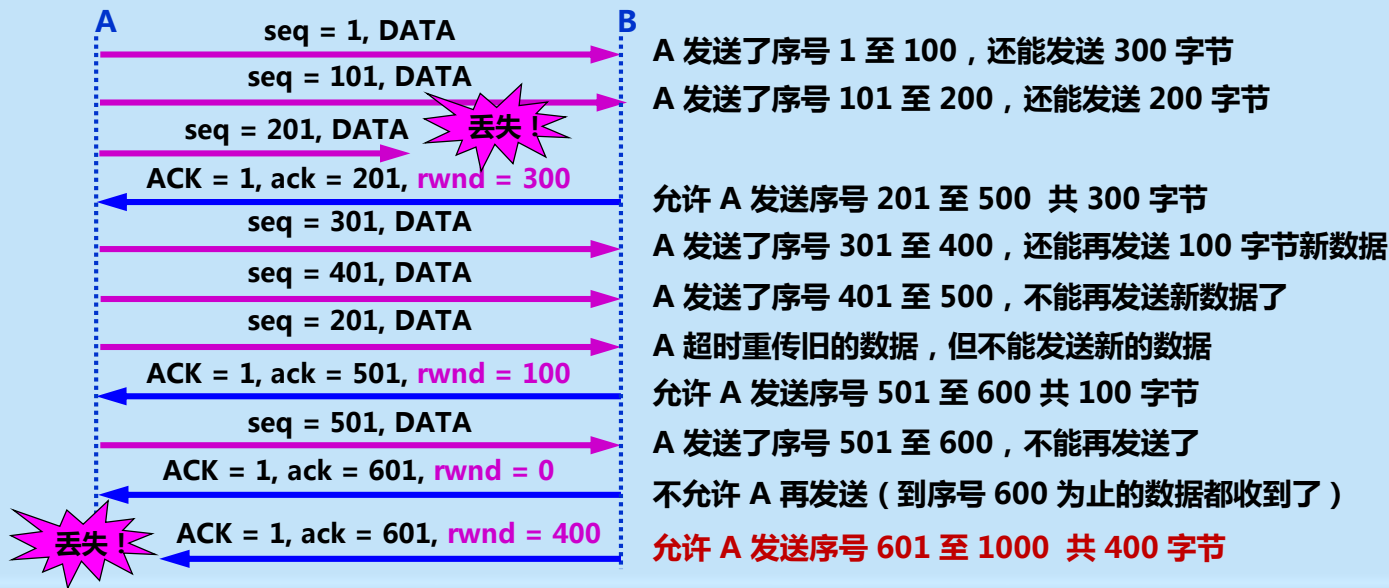
A 发送了序号 501 至 600，不能再发送了

不允许 A 再发送 (到序号 600 为止的数据都收到了)



利用可变窗口进行流量控制举例

A 向 B 发送数据。在连接建立时，B 告诉 A：
“我的接收窗口 $rwnd = 400$ (字节)”。





可能发⽣死锁

- B 向 A 发送了零窗口的报文段后不久，B 的接收缓存又有了一些存储空间。于是 B 向 A 发送了 $rwnd = 400$ 的报文段。
- 但这个报文段在传送过程中**丢失**了。A 一直等待收到 B 发送的非零窗口的通知，而 B 也一直等待 A 发送的数据。
- 如果没有其他措施，这种**互相等待的死锁**局面将一直延续下去。
- 为了解决这个问题，TCP 为每一个连接设有一个**持续计时器** (persistence timer)。



持续计时器

- 为了解决这个问题，TCP 为每一个连接设有一个**持续计时器** (persistence timer)。
- 只要 TCP 连接的一方收到对方的**零窗口**通知，就启动该持续计时器。
- 若持续计时器设置的时间到期，就发送一个**零窗口探测报文段**（仅携带 1 字节的数据），而对方就在确认这个探测报文段时给出了现在的窗口值。
- 若窗口仍然是零，则收到这个报文段的一方就重新设置持续计时器。
- 若窗口不是零，则死锁的僵局就可以打破了。



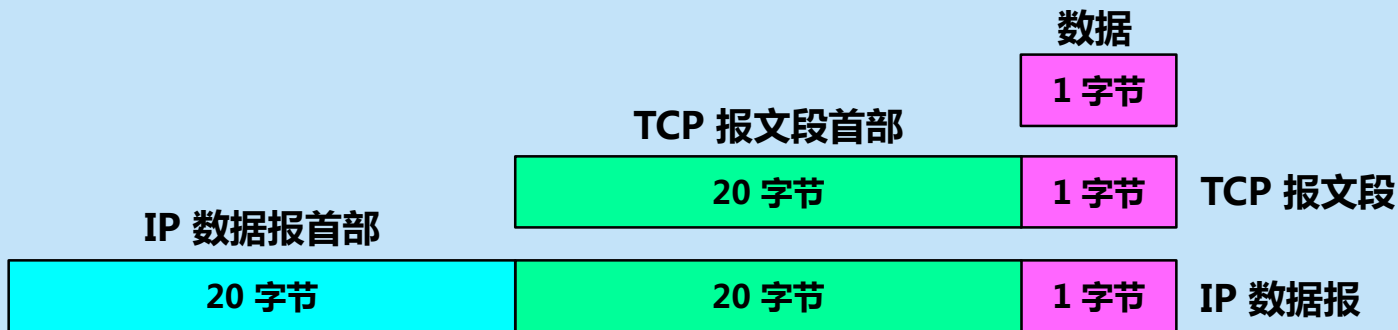
5.7.2 必须考虑传输效率

- 可以用不同的机制来控制 TCP 报文段的发送时机:
 1. **第一种机制**：TCP 维持一个变量 = **MSS**。只要缓存中存放的数据达到 MSS 字节时，就组装成一个 TCP 报文段发送出去。
 2. **第二种机制**：由发送方的应用进程指明要求发送报文段，即 TCP 支持的**推送** (push) 操作。
 3. **第三种机制**：发送方的一个**计时器**期限到了，这时就把当前已有的缓存数据装入报文段（但长度不能超过 MSS）发送出去。
- 如何控制 TCP 发送报文段的时机仍然是一个较为复杂的问题。



糊涂窗口综合症

- **糊涂窗口综合症**：每次仅发送一个字节或很少几个字节的数据时，有效数据传输效率变得很低的现象。





发送方糊涂窗口综合症

- 发送方 TCP 每次接收到一字节的数据后就发送。
- 这样，发送一个字节需要形成 41 字节长的 IP 数据报。效率很低。
- 解决方法：使用 Nagle 算法。



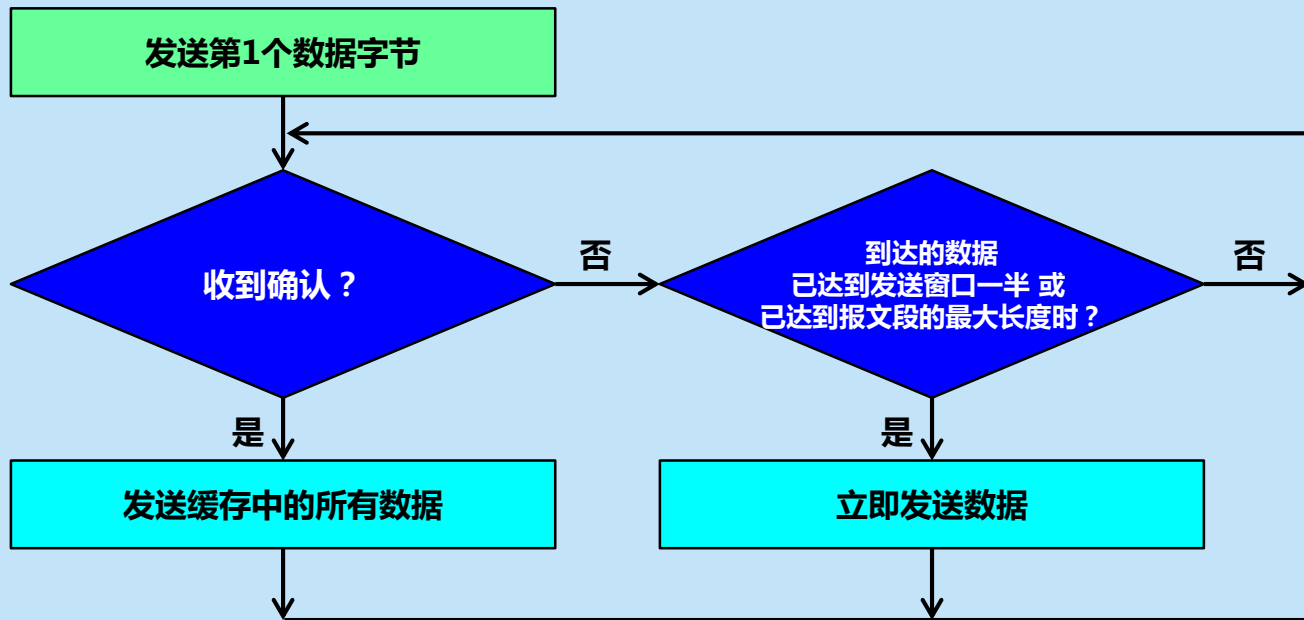
Nagle算法

若发送应用进程把要发送的数据逐个字节地送到 TCP 的发送缓存，则发送方就把**第一个数据字节先发送**出去，把后面到达的数据字节都缓存起来。

- 当发送方收到对**第一个数据字符的确认后**，再把发送缓存中的所有数据组装成一个报文段发送出去，同时继续对随后到达的数据进行缓存。
- 只有在收到对前一个报文段的**确认后**才继续发送下一个报文段。
- 当到达的数据已达到**发送窗口大小的一半**或已达到报文段的**最大长度**时，就立即发送一个报文段。



Nagle算法原理





接收方糊涂窗口综合症

- 当接收方的 TCP 缓冲区已满，接收方会向发送方发送窗口大小为 0 的报文。
- 若此时接收方的应用进程以交互方式每次只读取一个字节，于是接收方又发送窗口大小为一个字节的更新报文，发送方应邀发送一个字节的的数据（发送的 IP 数据报是 41 字节长），于是接收窗口又满了，如此循环往复。



接收方糊涂窗口综合症

原因：接收方应用进程消耗数据太慢，例如：每次只读取一个字节。



不允许 A 再发送（到序号 600 为止的数据都收到了）

应用进程读取 1 字节。接收缓存大小 = 1 字节

允许 A 发送序号 601 至 601 共 1 字节

A 发送了序号 601 至 601 字节，不能再发送了

应用进程读取 1 字节。接收缓存大小 = 1 字节

允许 A 发送序号 602 至 602 共 1 字节

A 发送了序号 602 至 602 字节，不能再发送了



接收方糊涂窗口综合症

解决方法：让接收方等待一段时间，只要出现这两种情况之一，接收方就发出确认报文，并向发送方通知当前的窗口大小。

- 接收缓存已有足够空间容纳一个最长的报文段。
- 到接收缓存已有一半空闲的空间。



5.8 TCP 的拥塞 控制

5.8.1

拥塞控制的一般原理

5.8.2

TCP 的拥塞控制方法

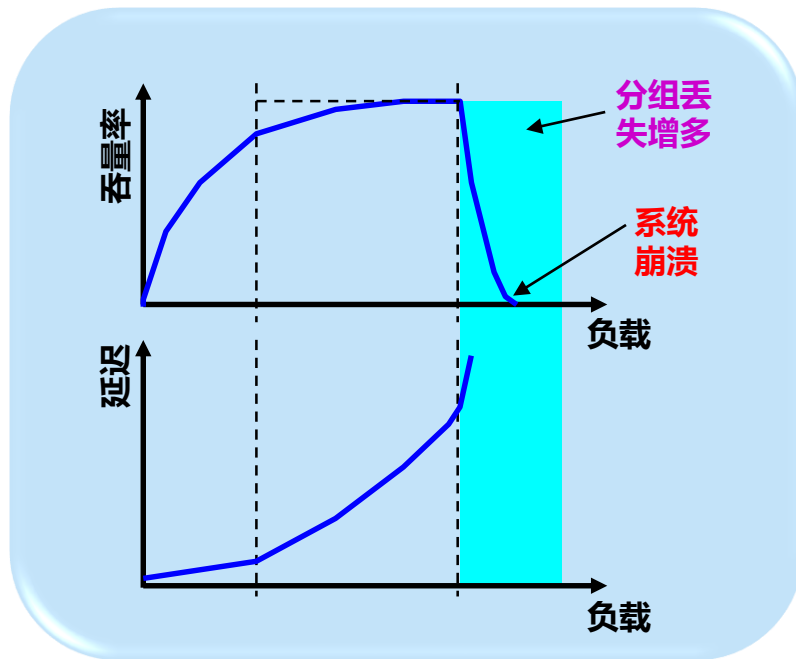
5.8.3

主动队列管理 AQM



5.8.1 拥塞控制的一般原理

- 在某段时间，若对网络中某资源的需求超过了该资源所能提供的可用部分，网络的性能就要变坏。这种现象称为**拥塞 (congestion)**。
- 最坏结果：**系统崩溃**。





拥塞产生的原因

- 网络拥塞往往是由许多因素引起的。例如：
 1. 节点缓存的容量太小；
 2. 链路的容量不足；
 3. 处理机处理的速率太慢；
 4. 拥塞本身会进一步加剧拥塞；
- 出现拥塞的原因：

$$\sum \text{对资源需求} > \text{可用资源} \quad (5-7)$$



增加资源能解决拥塞吗？

- **不能。**这是因为网络拥塞是一个非常复杂的问题。简单地采用上述做法，在许多情况下，不但不能解决拥塞问题，而且还可能使网络的性能更坏。
- 网络拥塞往往是由许多因素引起的。例如：
 1. 增大缓存，但未提高输出链路的容量和处理机的速度，排队等待时间将会大大增加，引起大量超时重传，解决不了网络拥塞；
 2. 提高处理机处理的速率会将瓶颈转移到其他地方；



拥塞控制与流量控制的区别

拥塞控制

防止过多的数据注入到网络中，使网络中的路由器或链路**不致过载**；

——是一个**全局性**的过程，涉及到与降低网络传输性能有关的所有因素。

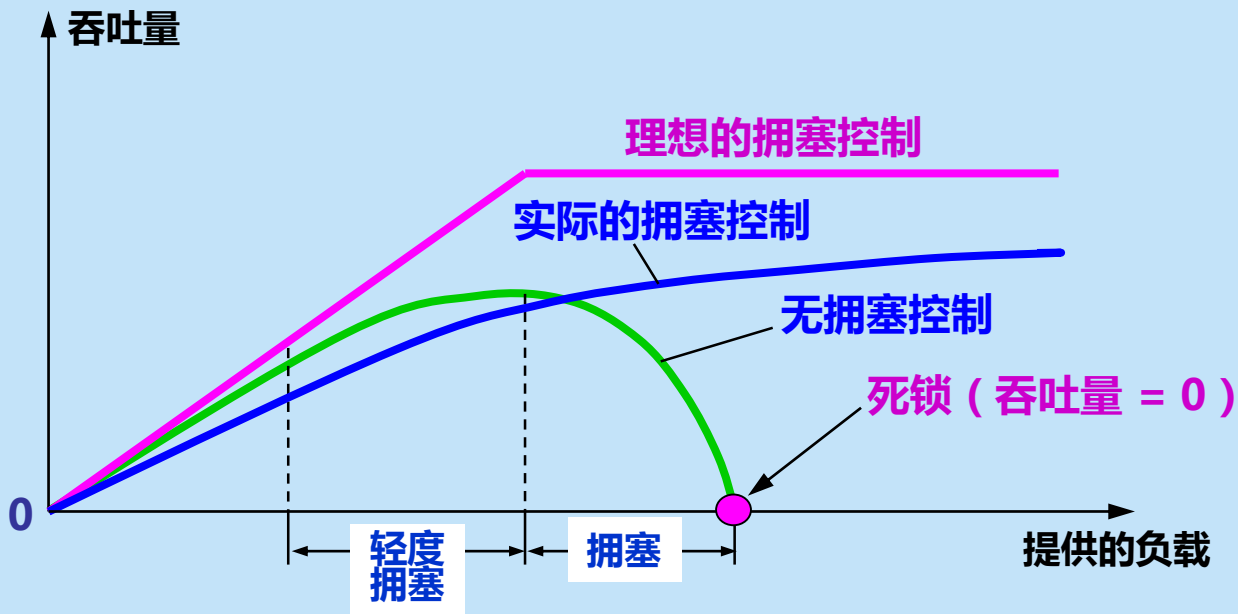
流量控制

抑制发送端发送数据的速率，以使接收端**来得及接收**；

——是点对点通信量的控制，是**端到端**的问题。



拥塞控制所起的作用





拥塞控制的一般原理

- 拥塞控制的前提：网络能够承受现有的网络负荷。
- 实践证明，拥塞控制是很难设计的，因为它是一个**动态问题**。
- 分组的丢失是网络发生拥塞的**征兆**而不是原因。
- 在许多情况下，甚至正是**拥塞控制本身**成为引起网络性能恶化、甚至发生死锁的原因。



开环控制和闭环控制

开环控制

在设计网络时，事先考虑周全，力求工作时不发生拥塞；

思路：力争避免发生拥塞。

闭环控制

基于反馈环路的概念；

根据网络当前的运行状态采取相应控制措施；

思路：在发生拥塞后，采取措施进行控制，消除拥塞。



闭环控制

属于闭环控制的有以下几种措施：

- (1) 监测网络系统，以便检测到拥塞在何时、何处发生。
- (2) 将拥塞发生的信息传送到可采取行动的地方。
- (3) 调整网络系统的运行以解决出现的问题。



监测网络的拥塞

- 主要指标有：
 1. 由于缺少缓存空间而被丢弃的分组的百分数；
 2. 平均队列长度；
 3. 超时重传的分组数；
 4. 平均分组时延；
 5. 分组时延的标准差，等等。
- 上述这些指标的上升都标志着拥塞的增长。



5.8.2 TCP 的拥塞控制方法

- TCP 采用**基于窗口的方法**进行拥塞控制。该方法属于闭环控制方法。
- TCP发送方维持一个**拥塞窗口 cwnd** (Congestion Window)
- 发送端利用**拥塞窗口**根据网络的拥塞情况调整发送的数据量。
- 发送窗口大小不仅取决于接收方窗口，还取决于网络的拥塞状况，
所以**真正的发送窗口值**为：

真正的发送窗口值 = $\text{Min}(\text{接收方窗口值}, \text{拥塞窗口值})$



控制拥塞窗口的原则

- **网络没有出现拥塞：**

拥塞窗口就可以再增大一些，以便把更多的分组发送出去，这样就可以提高网络的利用率。

- **网络出现拥塞或有可能出现拥塞：**

把拥塞窗口减小一些，以减少注入到网络中的分组数，以便缓解网络出现的拥塞。



拥塞的判断

重传定时器超时

网络已经发生了拥塞。

收到三个重复的 ACK

预示网络可能会出现拥塞（实际可能还未发生拥塞）。



TCP拥塞控制算法

- 四种拥塞控制算法 (RFC 5681) :
 - 慢开始 (slow-start)
 - 拥塞避免 (congestion avoidance)
 - 快重传 (fast retransmit)
 - 快恢复 (fast recovery)



慢开始 (Slow start)

- **目的**：用来确定网络的负载能力或拥塞程度。
- **算法的思路**：由小到大逐渐增大拥塞窗口数值。
- **两个变量**：

拥塞窗口

初始拥塞窗口值：2 种设置方法。

1 至 2 个最大报文段（旧标准）

2 至 4 个最大报文段（RFC 5681）

窗口值逐渐增大。

慢开始门限

防止拥塞窗口增长过大引起网络拥塞。



慢开始 (Slow start)

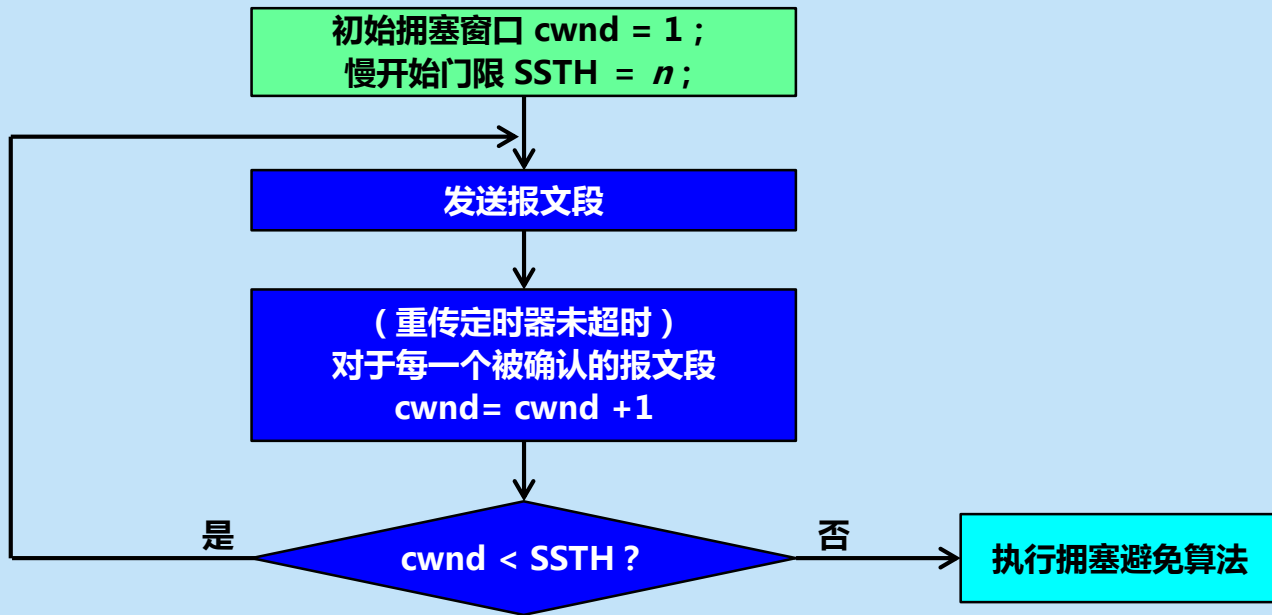
- **拥塞窗口 cwnd 控制方法**：在每收到一个**对新的报文段的确认后**，可以把拥塞窗口增加最多一个 SMSS 的数值。

$$\text{拥塞窗口 cwnd 每次的增加量} = \min(N, \text{SMSS}) \quad (5-8)$$

- 其中 N 是原先未被确认的、但现在被刚收到的确认报文段所确认的字节数。
- 不难看出，当 $N < \text{SMSS}$ 时，拥塞窗口每次的增加量要小于 SMSS。
- 用这样的方法逐步增大发送方的拥塞窗口 cwnd，可以使分组注入到网络的速率更加合理。

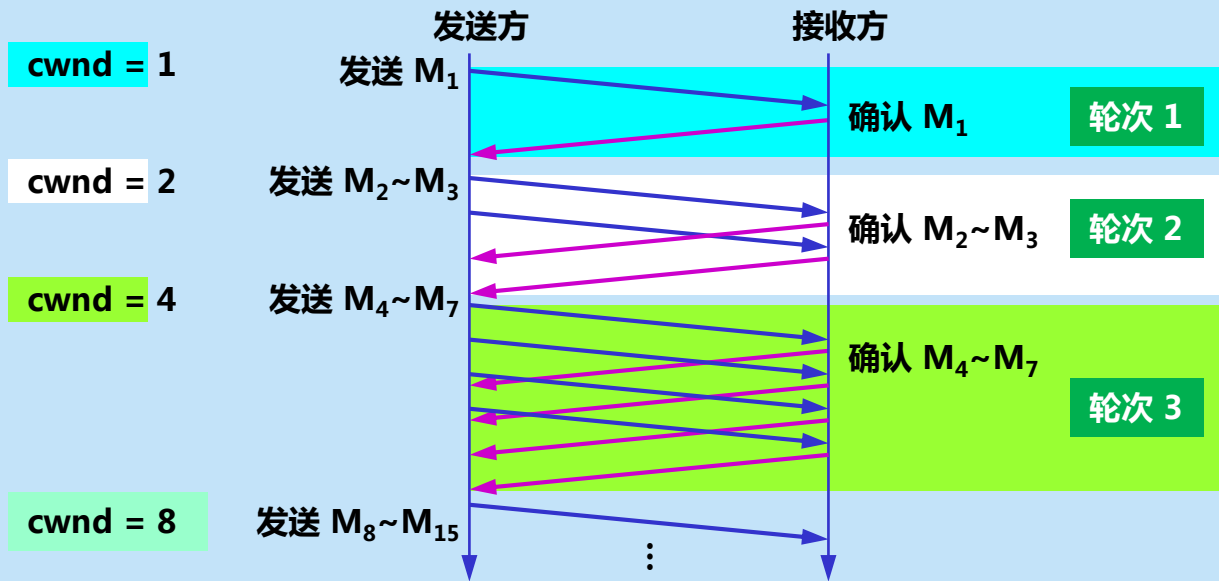


慢开始 (Slow start)



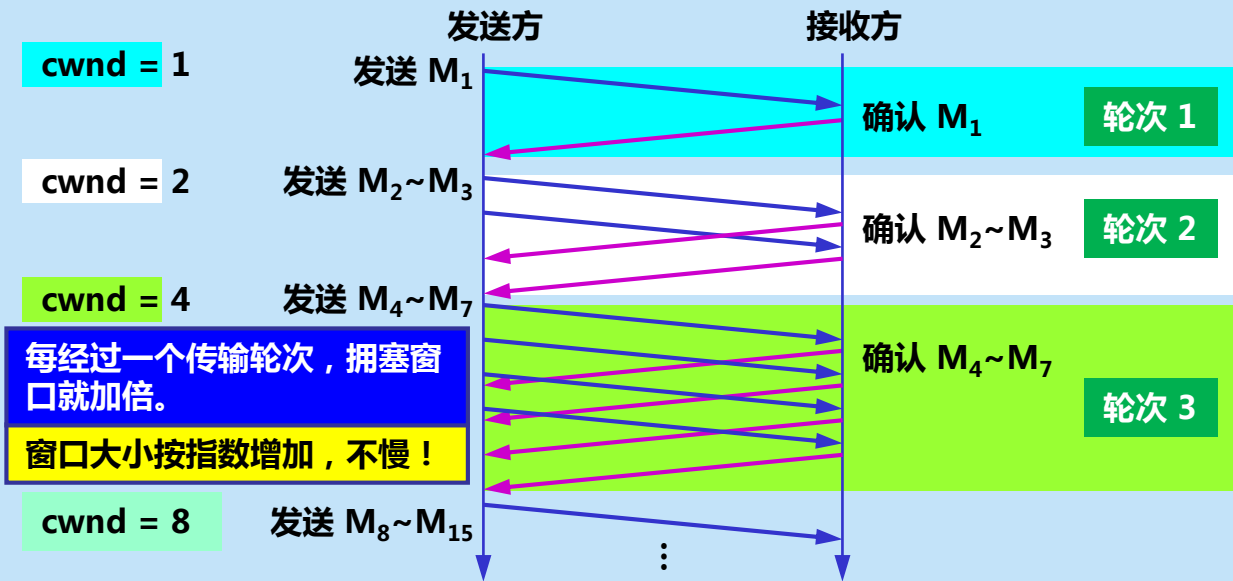


发送方每收到一个对新报文段的确认
(重传的不算在内) 就使 cwnd 加 1。





发送方每收到一个对新报文段的确认
(重传的不算在内) 就使 cwnd 加 1。





传输轮次

- 使用慢开始算法后，每经过一个**传输轮次** (transmission round)，拥塞窗口 $cwnd$ 就加倍。
- 一个传输轮次所经历的时间其实就是往返时间 RTT。
- “**传输轮次**” 更加强调：把拥塞窗口 $cwnd$ 所允许发送的报文段都连续发送出去，并收到了对已发送的最后一个字节的确认。
- 例如，拥塞窗口 $cwnd = 4$ ，这时的往返时间 RTT 就是发送方连续发送 4 个报文段，并收到这 4 个报文段的确认，总共经历的时间。



设置慢开始门限状态变量 ssthresh

- 慢开始门限 ssthresh 的用法如下：
 1. 当 $cwnd < ssthresh$ 时，使用慢开始算法。
 2. 当 $cwnd > ssthresh$ 时，使用拥塞避免算法。
 3. 当 $cwnd = ssthresh$ 时，慢开始算法/拥塞避免算法。

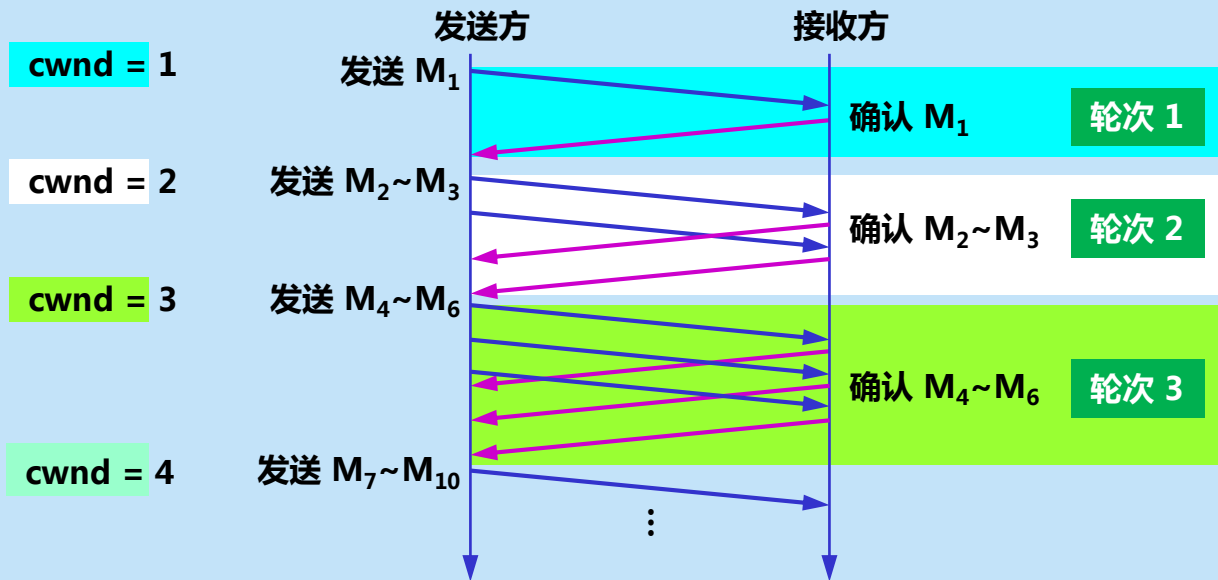


拥塞避免算法

- **思路**：让拥塞窗口 $cwnd$ **缓慢地增大**，避免出现拥塞。
- 每经过一个传输轮次，拥塞窗口 $cwnd = cwnd + 1$ 。
- 使拥塞窗口 $cwnd$ **按线性规律缓慢增长**。
- 在拥塞避免阶段，具有 **“加法增大”** (Additive Increase) 的特点。



在超时之前，每经过一个传输轮次就使 cwnd 加 1。



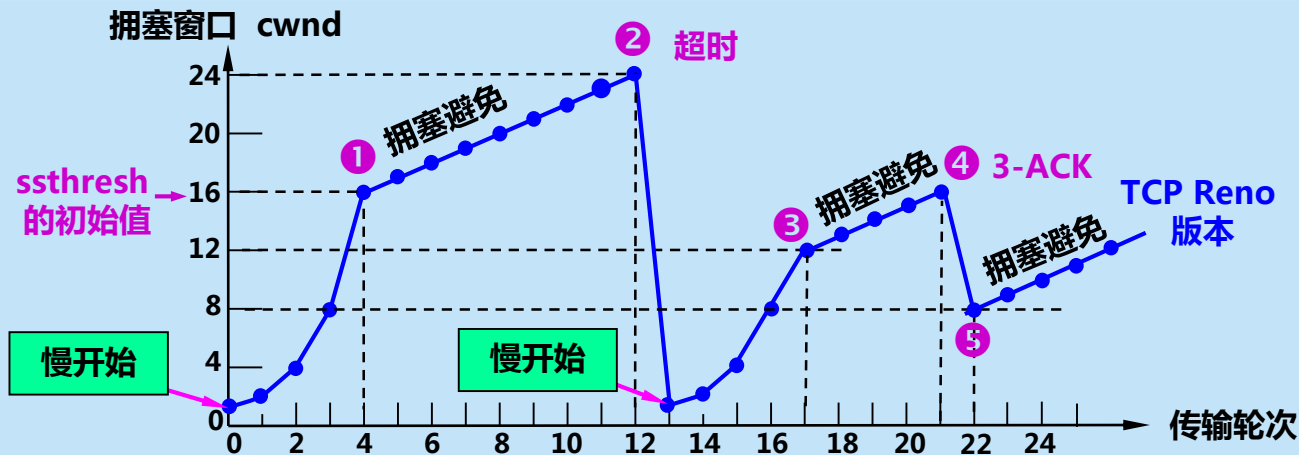


当网络出现拥塞时

- 无论在慢开始阶段还是在拥塞避免阶段，只要发送方判断网络出现拥塞（重传定时器超时）：
 1. $ssthresh = \max(cwnd/2, 2)$
 2. $cwnd = 1$
 3. 执行慢开始算法
- 目的：迅速减少主机发送到网络中的分组数，使得发生拥塞的路由器有足够时间把队列中积压的分组处理完毕。



慢开始和拥塞避免算法的实现举例

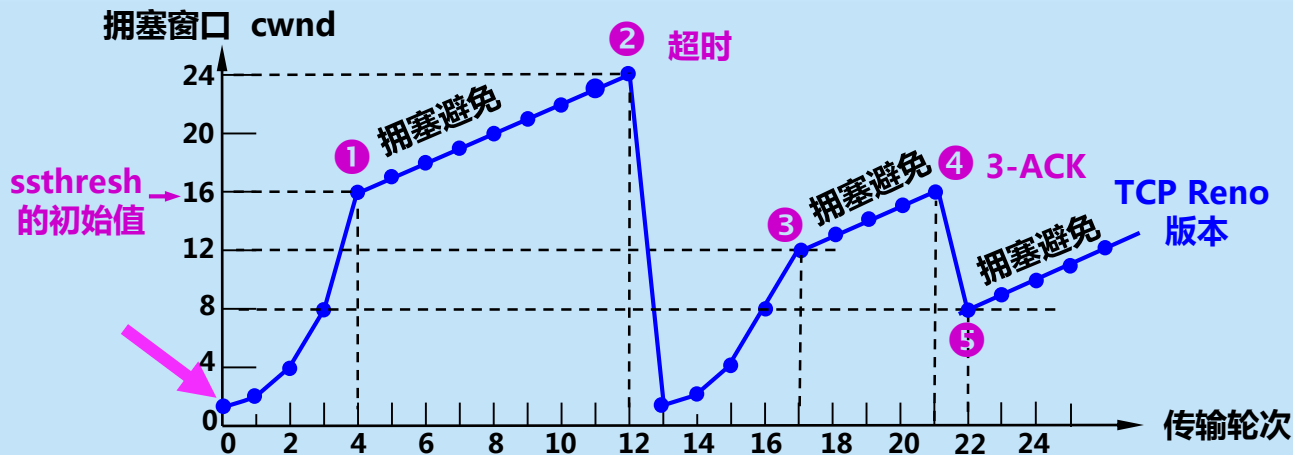


当 TCP 连接进行初始化时，将拥塞窗口置为 1。图中的窗口单位不使用字节而使用**报文段**。

慢开始门限的初始值设置为 16 个报文段，即 $ssthresh = 16$ 。



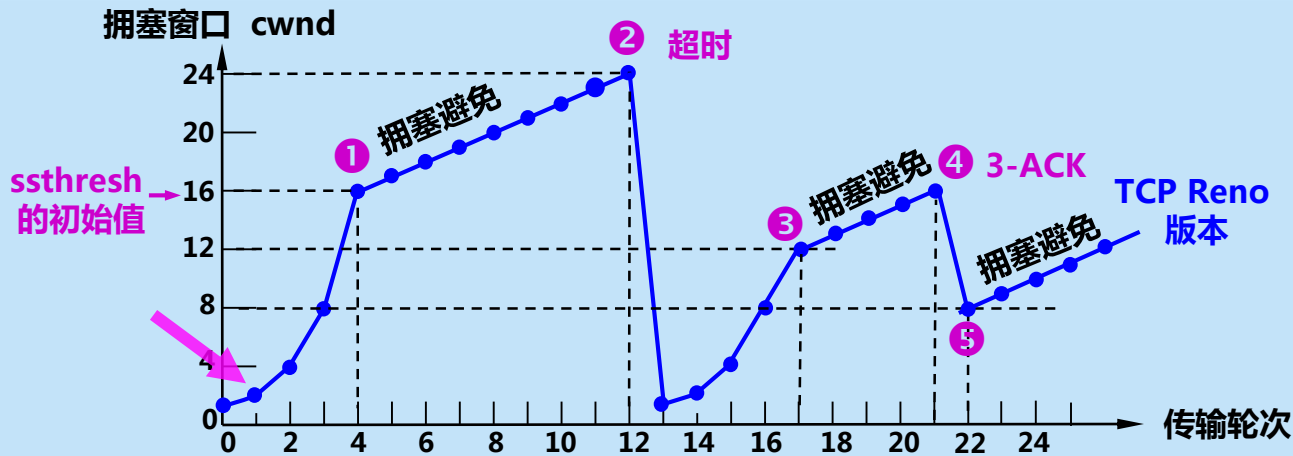
慢开始和拥塞避免算法的实现举例



在执行慢开始算法时，拥塞窗口 $cwnd=1$ ，发送第一个报文段。



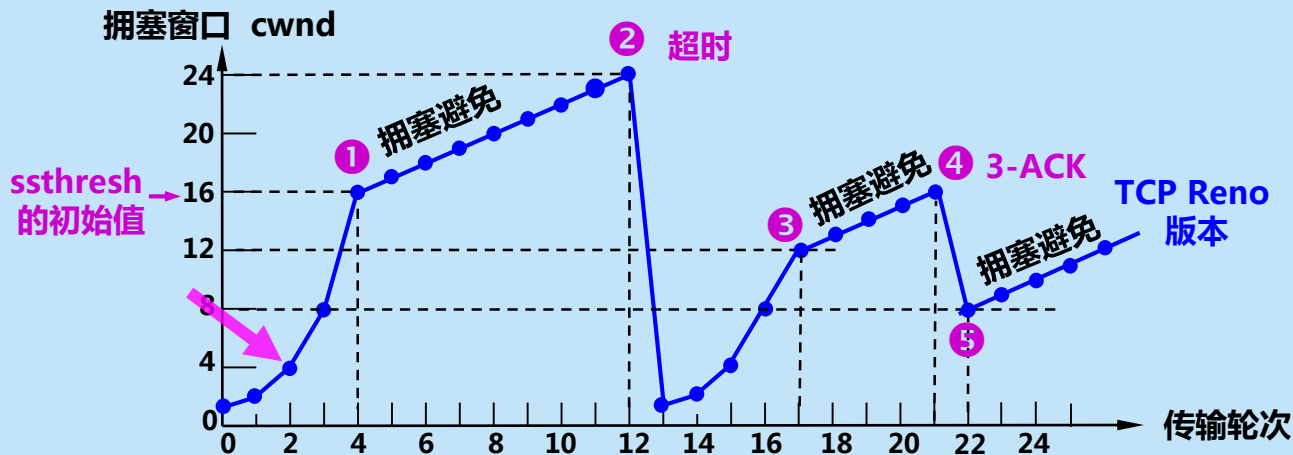
慢开始和拥塞避免算法的实现举例



发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1，然后开始下一轮的传输（请注意，横坐标是传输轮次，不是时间）。因此拥塞窗口 cwnd 随着传输轮次按指数规律增长。



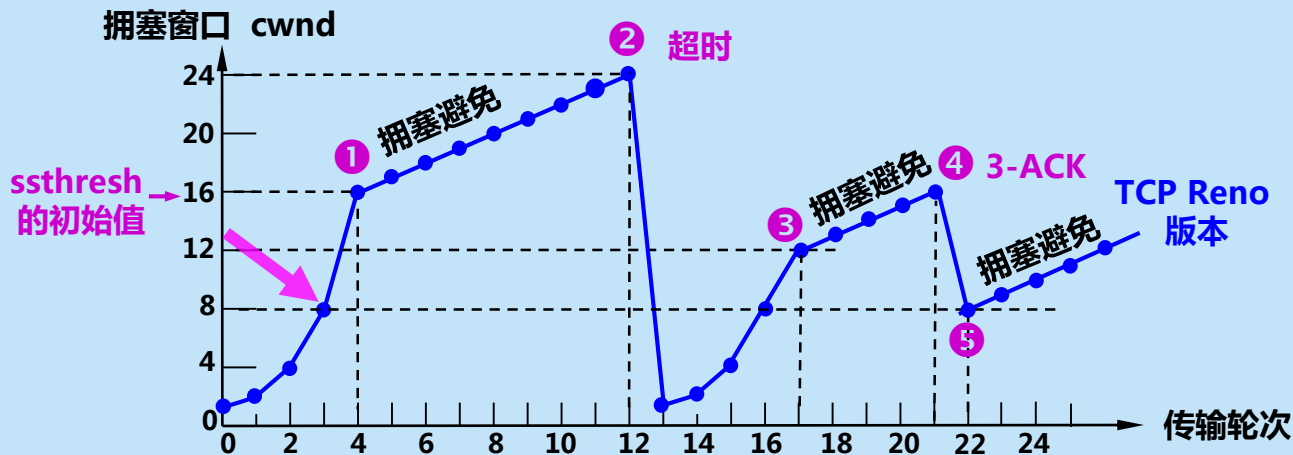
慢开始和拥塞避免算法的实现举例



发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1，然后开始下一轮的传输（请注意，横坐标是传输轮次，不是时间）。因此拥塞窗口 cwnd 随着传输轮次按指数规律增长。



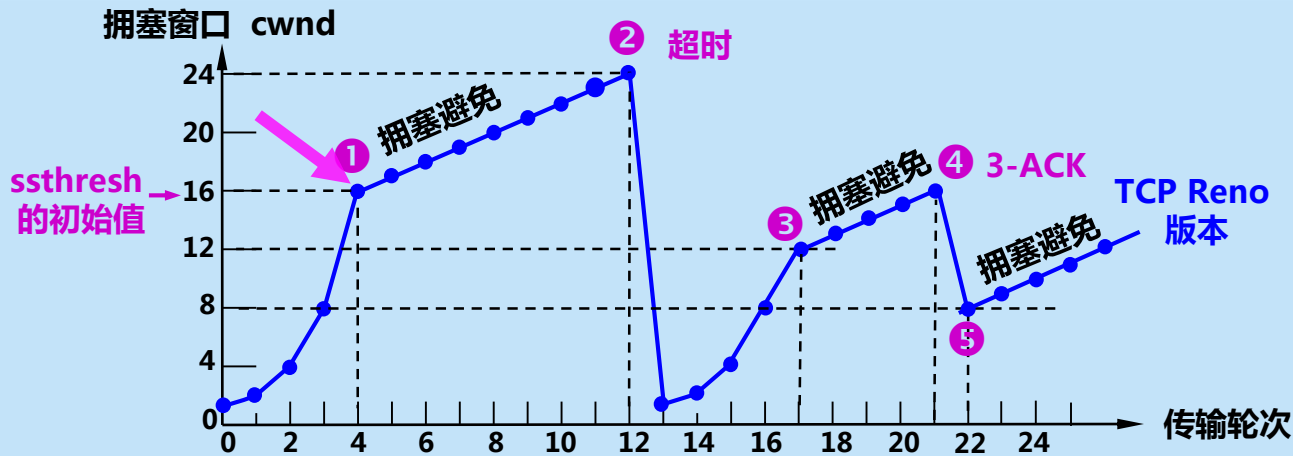
慢开始和拥塞避免算法的实现举例



发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1，然后开始下一轮的传输（请注意，横坐标是传输轮次，不是时间）。因此拥塞窗口 cwnd 随着传输轮次按指数规律增长。



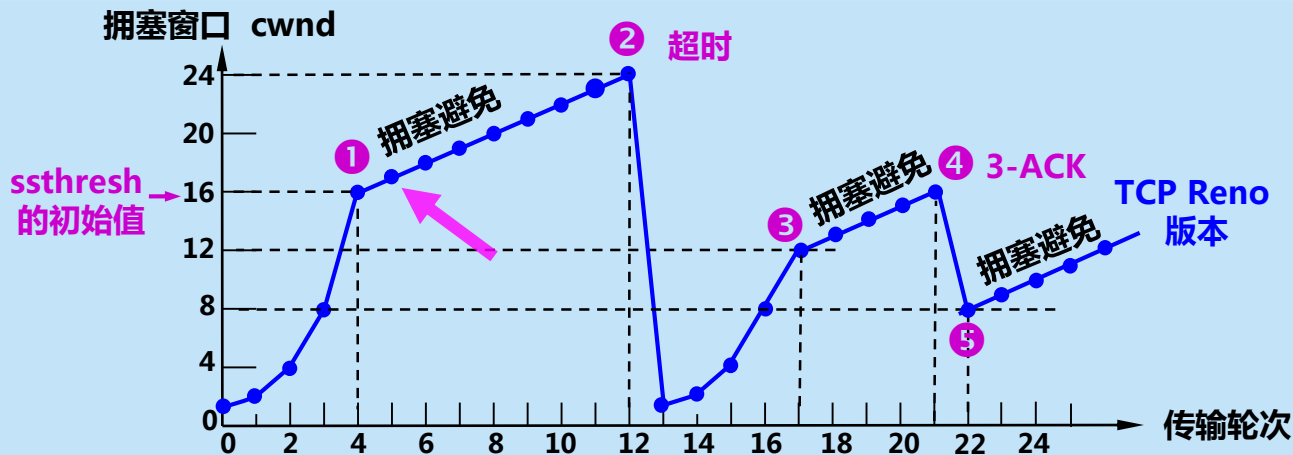
慢开始和拥塞避免算法的实现举例



当拥塞窗口 cwnd 增长到慢开始门限值 ssthresh 时 (图中的点① , 此时拥塞窗口 cwnd = 16) , 就改为执行拥塞避免算法 , 拥塞窗口按线性规律增长。



慢开始和拥塞避免算法的实现举例



当拥塞窗口 cwnd 增长到慢开始门限值 ssthresh 时 (图中的点① , 此时拥塞窗口 cwnd = 16) , 就改为执行拥塞避免算法 , 拥塞窗口按线性规律增长。

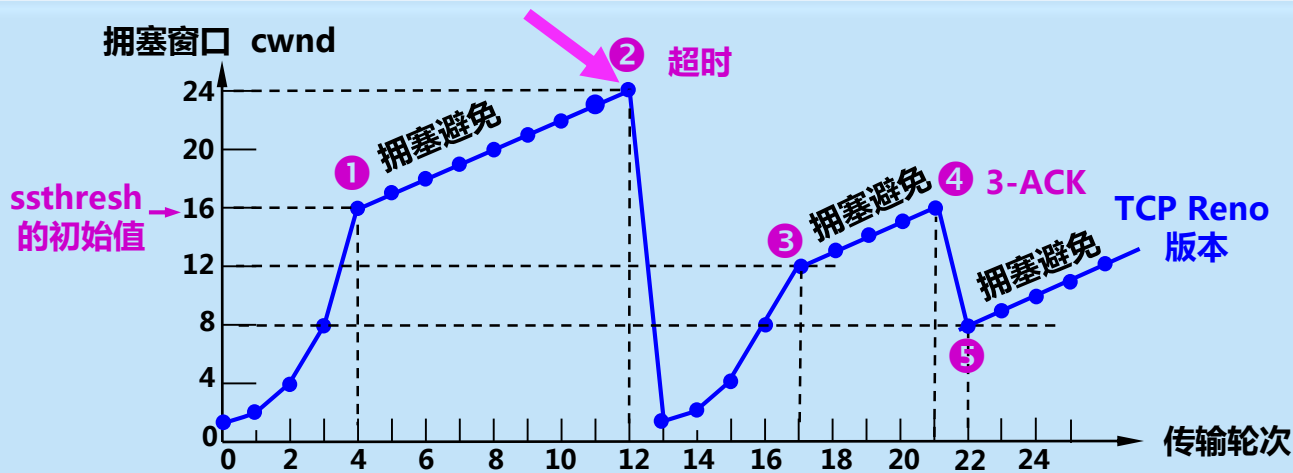


必须强调指出

- “拥塞避免”并非指完全能够避免了拥塞。利用以上的措施要完全避免网络拥塞还是不可能的。
- “拥塞避免”是说在拥塞避免阶段把拥塞窗口控制为按线性规律增长，使网络比较不容易出现拥塞。



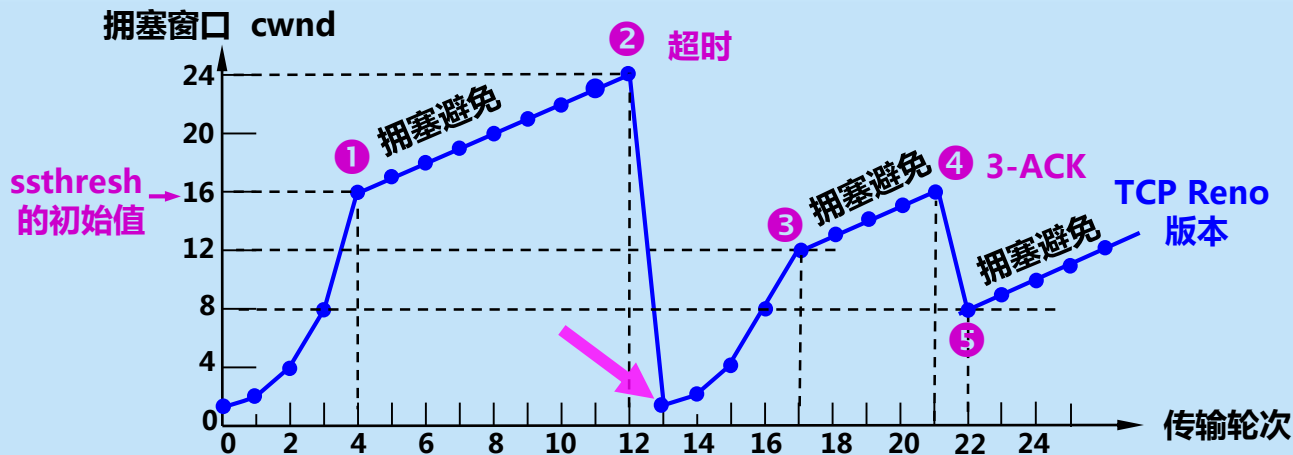
慢开始和拥塞避免算法的实现举例



当拥塞窗口 $cwnd = 24$ 时，网络出现了超时（图中的点②），发送方判断为网络拥塞。于是调整门限值 $sssthresh = cwnd / 2 = 12$ ，同时设置拥塞窗口 $cwnd = 1$ ，进入慢开始阶段。



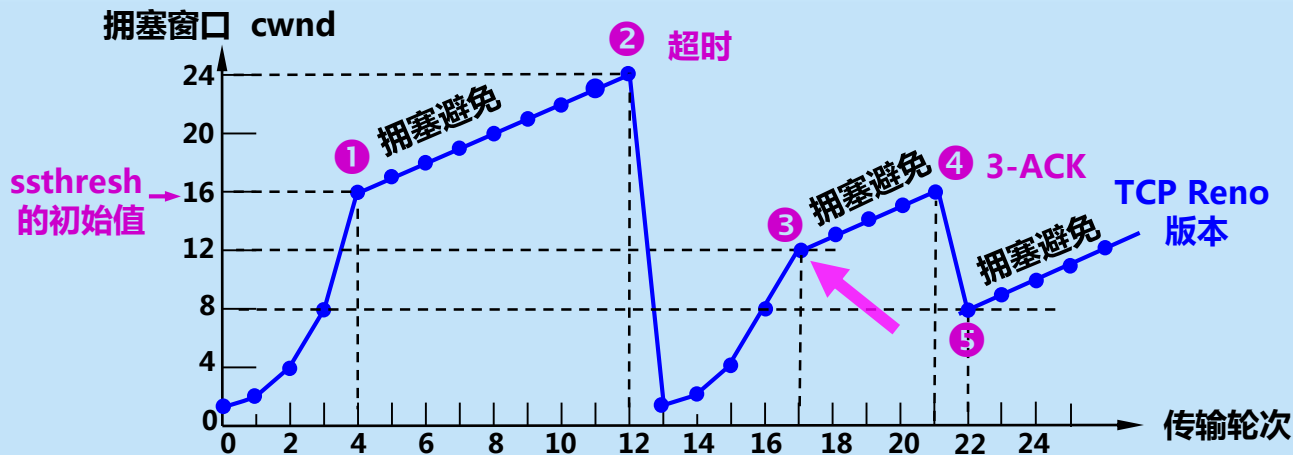
慢开始和拥塞避免算法的实现举例



当拥塞窗口 $cwnd = 24$ 时，网络出现了超时（图中的点②），发送方判断为网络拥塞。于是调整门限值 $sssthresh = cwnd / 2 = 12$ ，同时设置拥塞窗口 $cwnd = 1$ ，进入慢开始阶段。



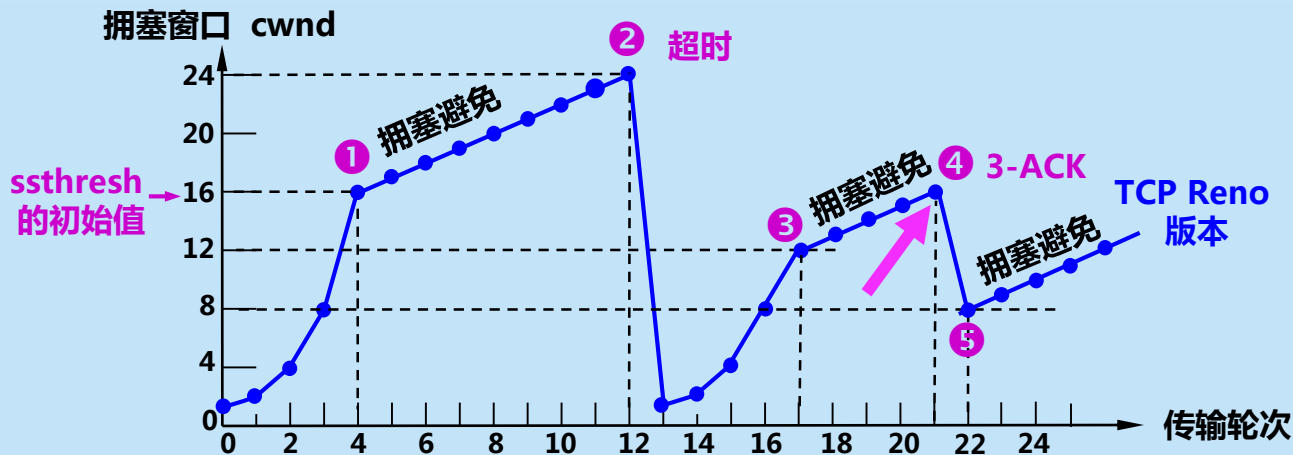
慢开始和拥塞避免算法的实现举例



按照慢开始算法，发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1。当拥塞窗口 $cwnd = ssthresh = 12$ 时（图中的点③，这是新的 $ssthresh$ 值），改为执行拥塞避免算法，拥塞窗口按线性规律增大。



慢开始和拥塞避免算法的实现举例



当拥塞窗口 $cwnd = 16$ 时 (图中的点④), 出现了一个新的情况, 就是发送方一连收到 3 个对同一个报文段的重复确认 (图中记为 3-ACK)。发送方改为执行快重传和快恢复算法。



快重传算法

- 发送方只要一连收到三个重复确认，就知道接收方确实没有收到报文段，因而应当立即进行重传（即“快重传”），这样就不会出现超时，发送方也不就会误认为出现了网络拥塞。
- 使用快重传可以使整个网络的吞吐量提高约20%。

不难看出，快重传并非取消重传计时器，而是在某些情况下可以更早地（更快地）重传丢失的报文段。

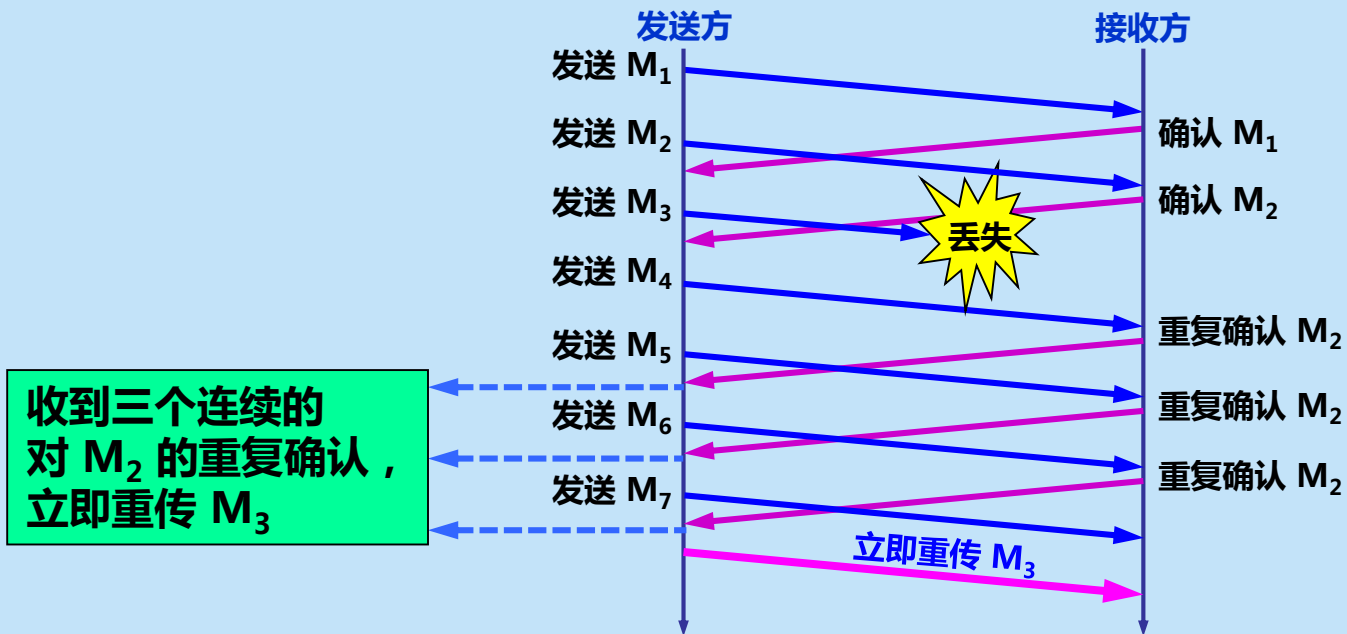


快重传算法

- 采用**快重传** FR (Fast Retransmission) 算法可以让发送方**尽早知道**发生了个别报文段的丢失。
- **快重传** 算法首先要求接收方不要等待自己发送数据时才进行捎带确认，而是要立即发送确认，即使收到了失序的报文段也要立即发出对已收到的报文段的重复确认。



快重传举例



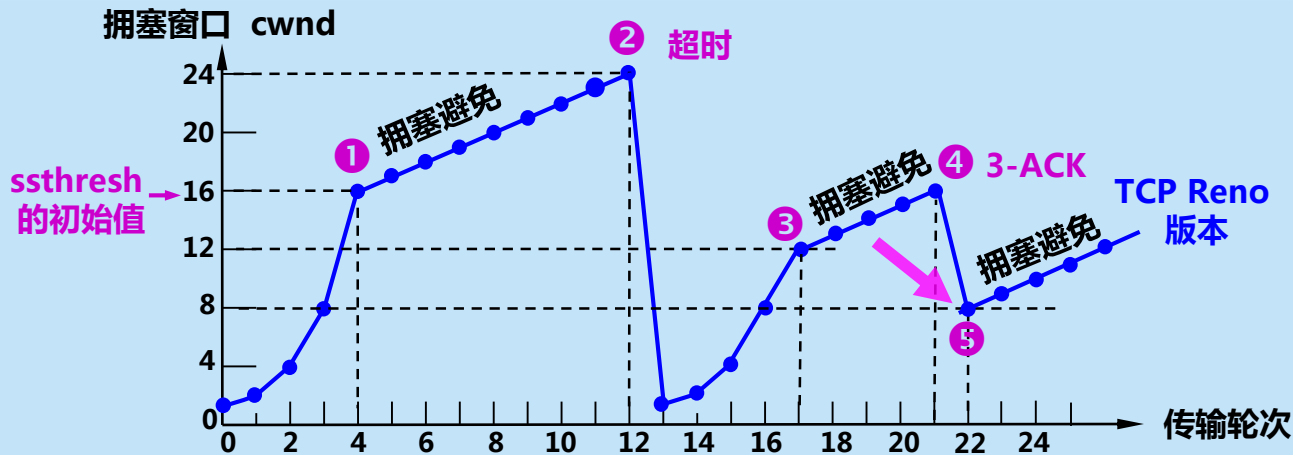


快恢复算法

- 当发送端收到连续三个重复的确认时，由于发送方现在认为网络很可能没有发生拥塞，因此现在**不执行慢开始算法**，而是执行**快恢复算法 FR (Fast Recovery) 算法**：
 1. 慢开始门限 $ssthresh = \text{当前拥塞窗口 } cwnd / 2$ ；
 2. 新拥塞窗口 $cwnd = \text{慢开始门限 } ssthresh$ ；
 3. 开始执行拥塞避免算法，使拥塞窗口缓慢地线性增大。



慢开始和拥塞避免算法的实现举例



因此，在图的点④，发送方知道现在只是丢失了个别的报文段。于是不启动慢开始，而是执行快恢复算法。这时，发送方调整门限值 $ssthresh = cwnd / 2 = 8$ ，同时设置拥塞窗口 $cwnd = ssthresh = 8$ （见图中的点⑤），并开始执行拥塞避免算法。

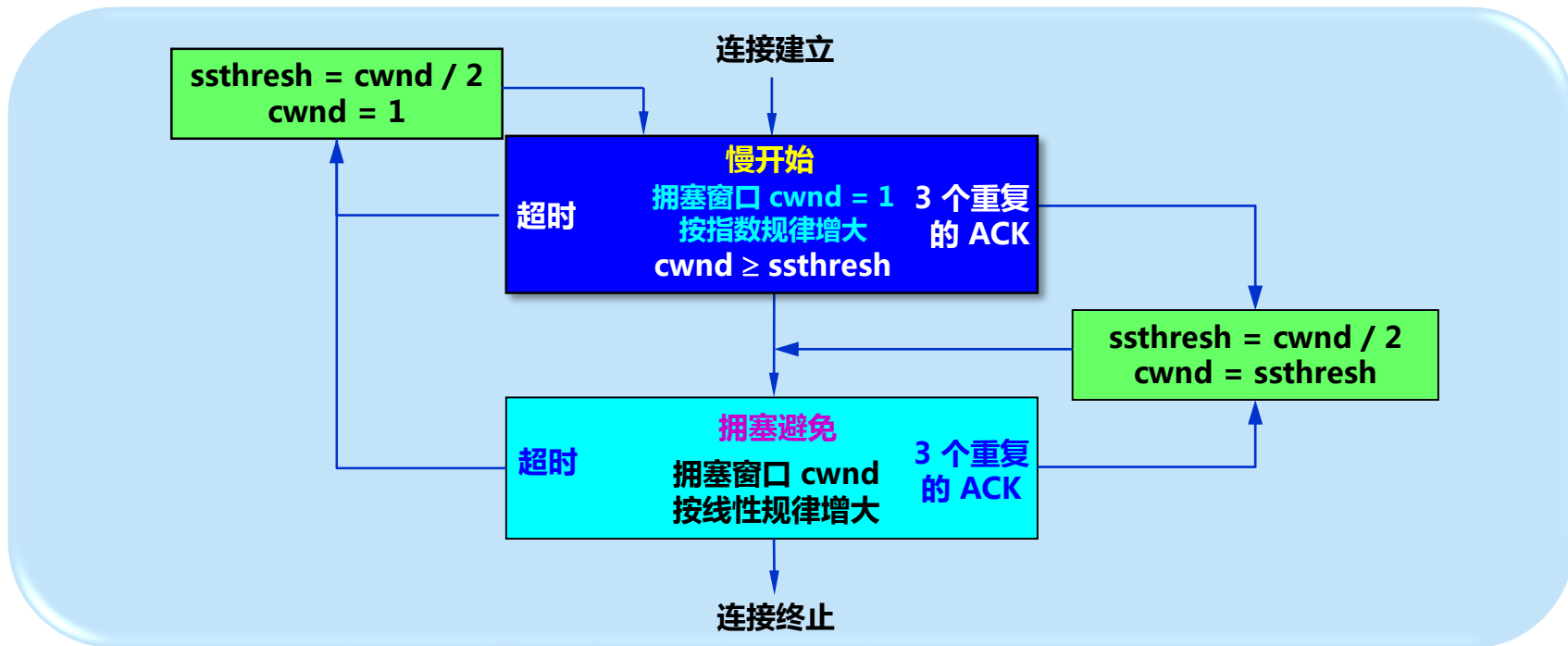


加法增大，乘法减小 (AIMD)

- 可以看出，在拥塞避免阶段，拥塞窗口是按照线性规律增大的。这常称为“**加法增大**” AI (Additive Increase)。
- 当出现超时或3个重复的确认时，就要把门限值设置为当前拥塞窗口值的一半，并大大减小拥塞窗口的数值。这常称为“**乘法减小**” MD (Multiplicative Decrease)。
- 二者合在一起就是所谓的 AIMD 算法。



TCP拥塞控制流程图





发送窗口的上限值

- 发送方的发送窗口的上限值应当取为接收方窗口 $rwnd$ 和拥塞窗口 $cwnd$ 这两个变量中较小的一个，即应按以下公式确定：

$$\text{发送窗口的上限值} = \text{Min} [rwnd, cwnd] \quad (5-9)$$

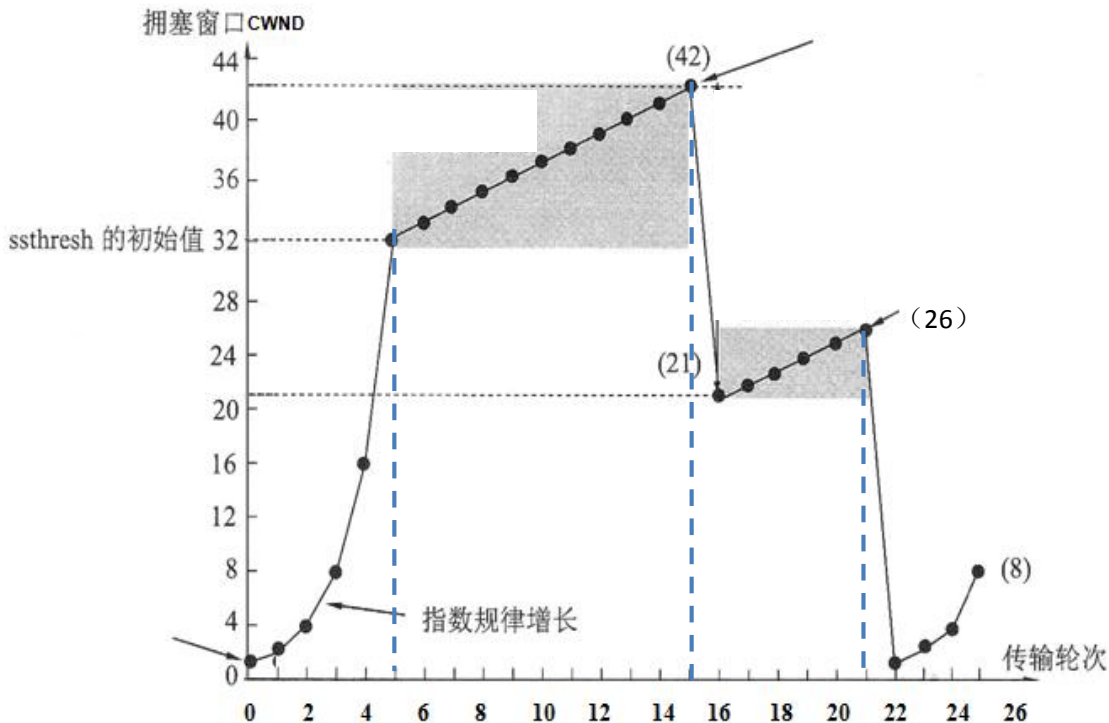
- 当 $rwnd < cwnd$ 时，是接收方的接收能力限制发送窗口的最大值。
- 当 $cwnd < rwnd$ 时，则是网络的拥塞限制发送窗口的最大值。

也就是说， $rwnd$ 和 $cwnd$ 中数值较小的一个，控制了发送方发送数据的速率。



【例1】TCP拥塞控制

1. 慢开始阶段时间？
2. 拥塞避免阶段时间？
3. 15轮次和21轮次如何发现报文段丢失？
4. 1轮次，18轮次、24轮次发送时，ssthresh多大？
5. 第几轮次发出70个报文段？
6. 假定25轮次后收到3个重复确认，检测出报文段丢失，那么拥塞窗口和ssthresh应设多大？





5.9 TCP 的运输 连接管理

5.9.1

TCP 的连接建立

5.9.2

TCP 的连接释放

5.9.3

TCP 的有限状态机



运输连接的三个阶段

- TCP 是面向连接的协议。
- TCP 连接有三个阶段：
 1. 连接建立
 2. 数据传送
 3. 连接释放
- TCP 连接的管理就是使 TCP 连接的建立和释放都能正常地进行。



TCP 连接建立过程中要解决的三个问题

1. 要使每一方能够确知对方的存在。
2. 要允许双方协商一些参数
 - 最大窗口值、是否使用窗口扩大选项
 - 时间戳选项
 - 服务质量
3. 能够对运输实体资源（如缓存大小、连接表中的项目等）进行分配。



客户—服务器方式

- TCP 连接的建立**采用客户服务器方式**。
- 主动发起连接建立的应用进程叫做**客户** (client)。
- 被动等待连接建立的应用进程叫做**服务器** (server)。



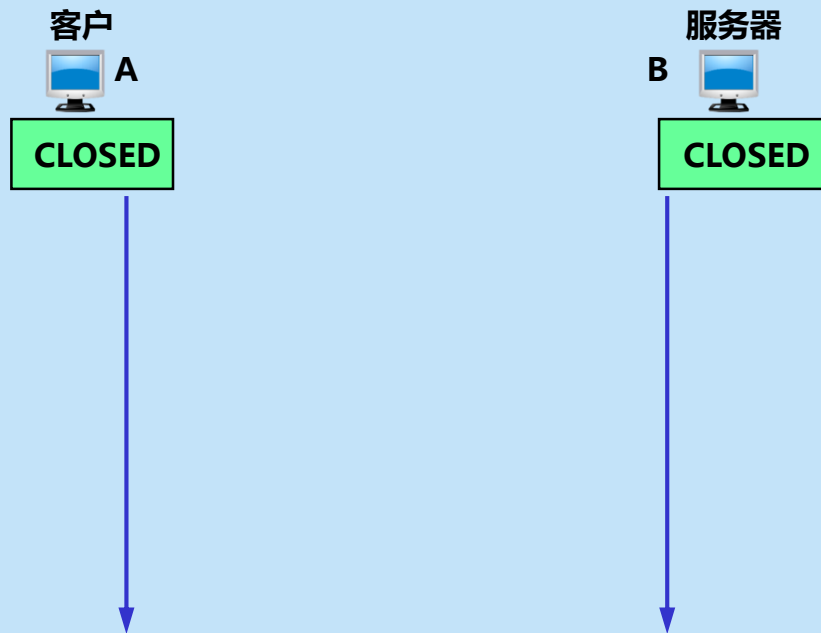
5.9.1 TCP 的连接建立

- TCP 建立连接的过程叫做**握手**。
- 握手需要在客户和服务端之间交换三个 TCP 报文段。称之为**三报文握手**。
- 采用**三报文握手**主要是为了防止已失效的连接请求报文段突然又传送到了，因而产生错误。



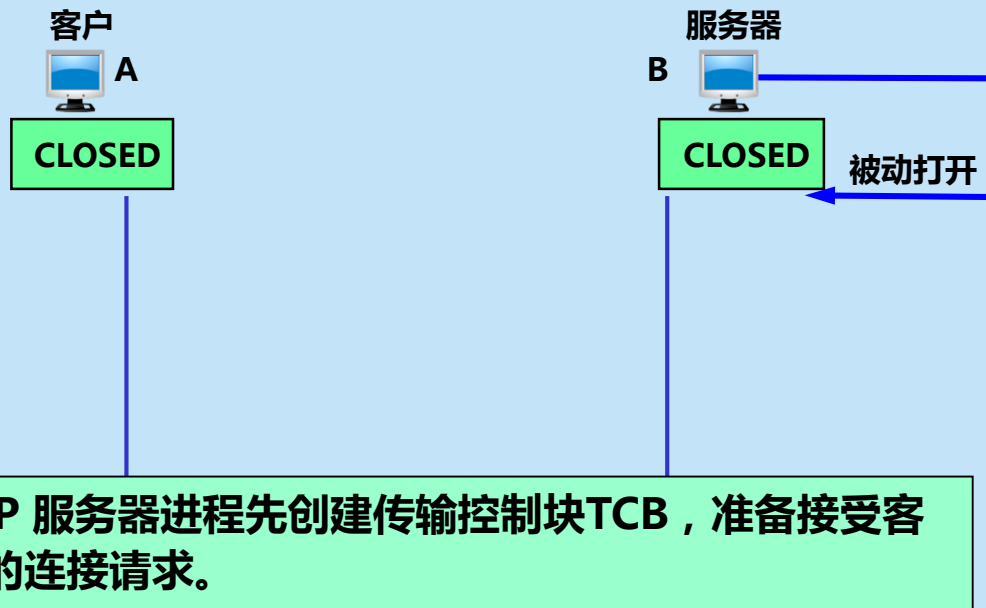


TCP 的连接建立：采用三报文握手



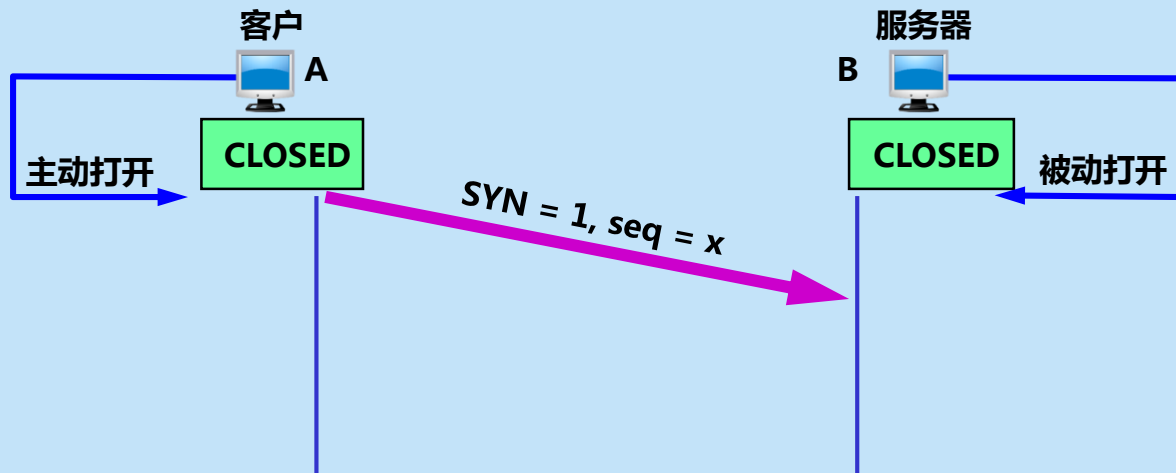


TCP 的连接建立：采用三报文握手





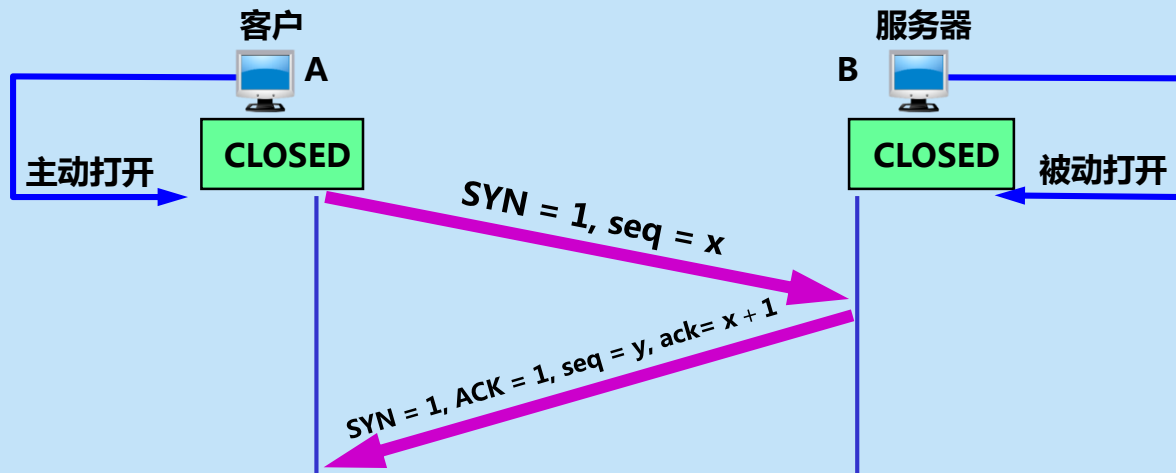
TCP 的连接建立：采用三报文握手



A 的 TCP 向 B 发出连接请求报文段，其首部中的同步位 $SYN = 1$ ，并选择序号 $seq = x$ ，表明传送数据时的第一个数据字节的序号是 x 。



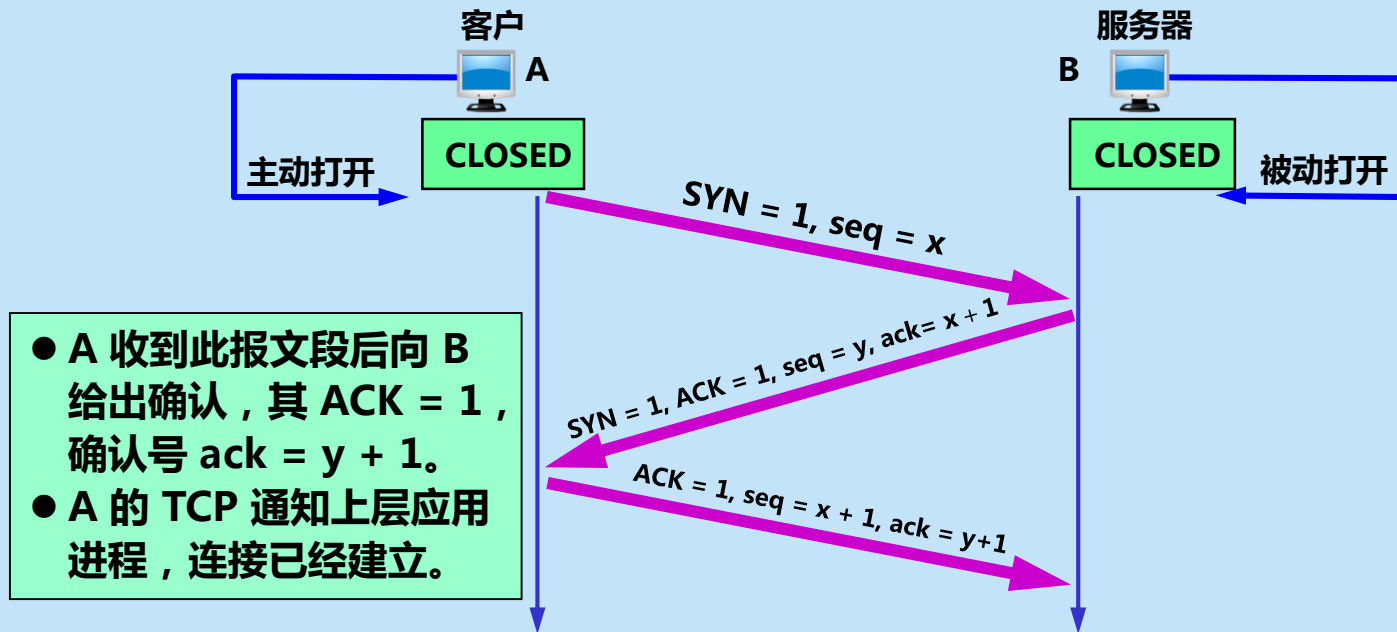
TCP 的连接建立：采用三报文握手



- B 的 TCP 收到连接请求报文段后，如同意，则发回确认。
- B 在确认报文段中应使 $SYN = 1$ ，使 $ACK = 1$ ，其确认号 $ack = x + 1$ ，自己选择的序号 $seq = y$ 。

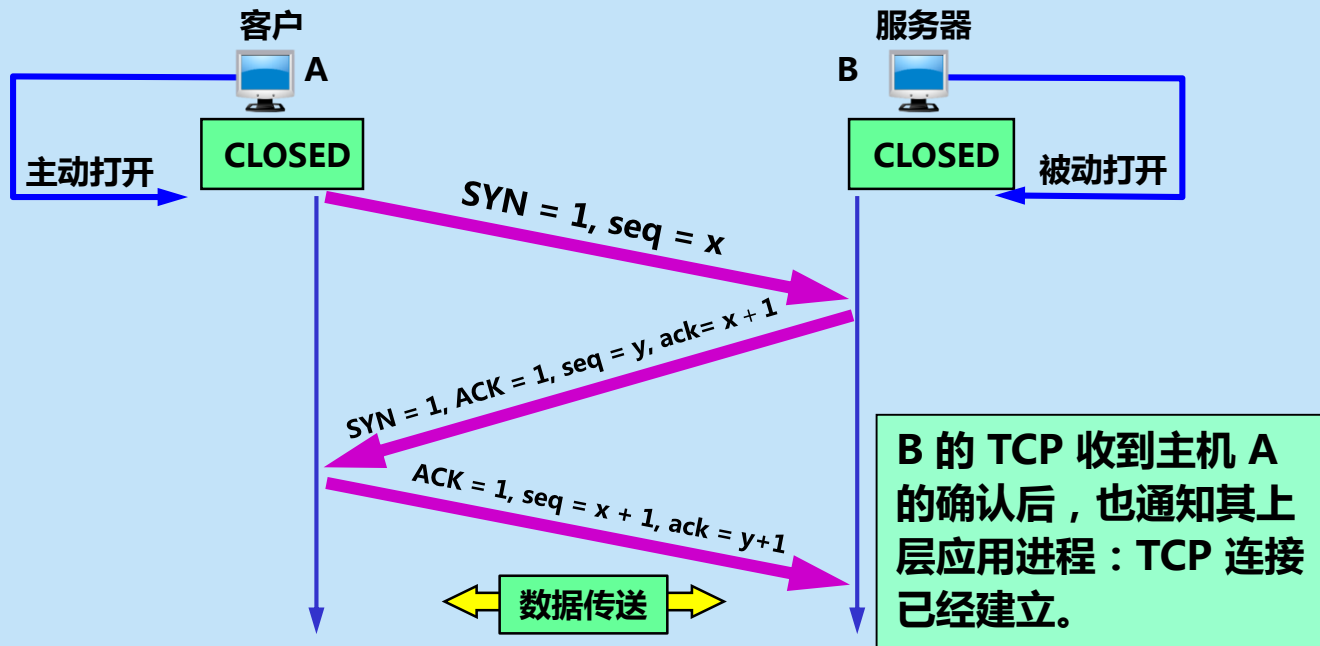


TCP 的连接建立：采用三报文握手



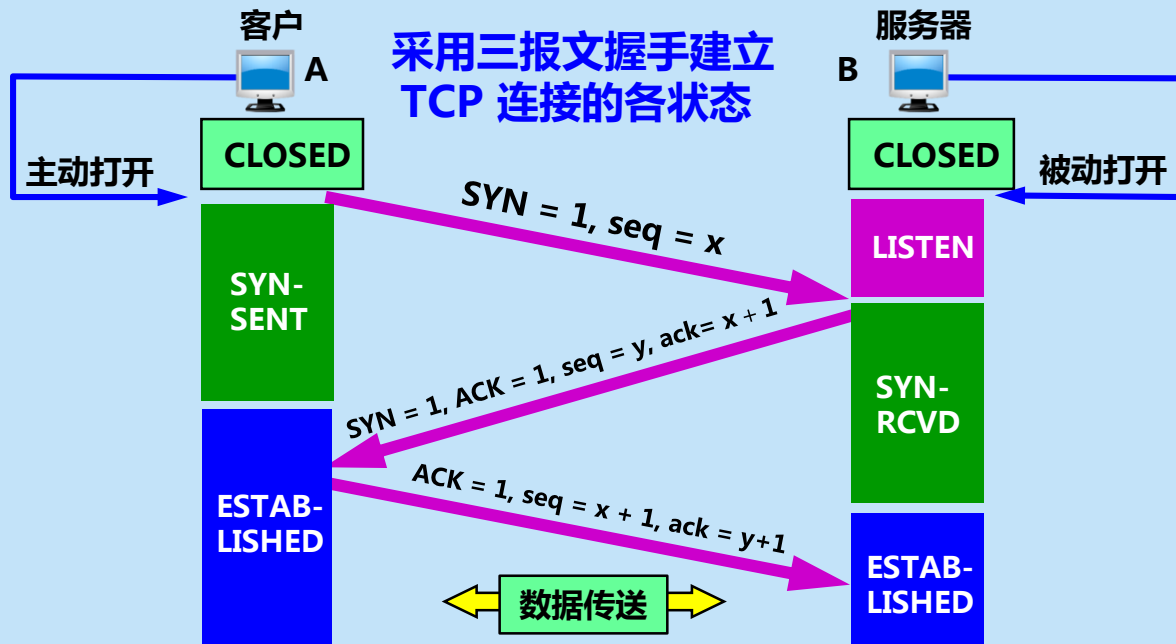


TCP 的连接建立：采用三报文握手





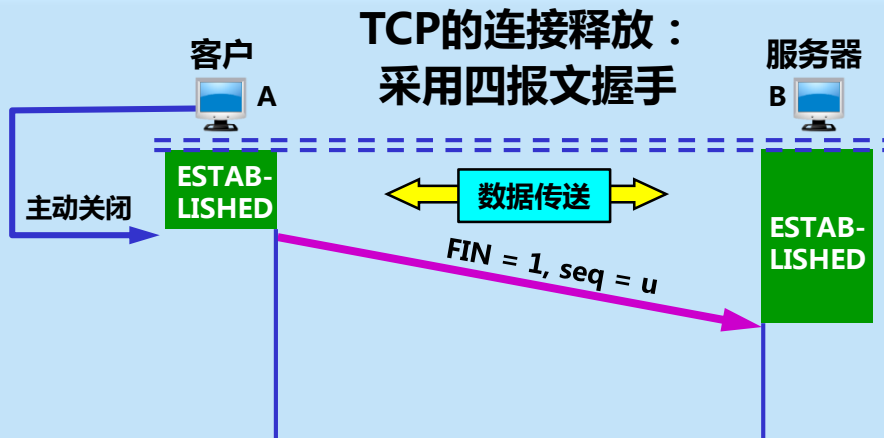
TCP 的连接建立：采用三报文握手



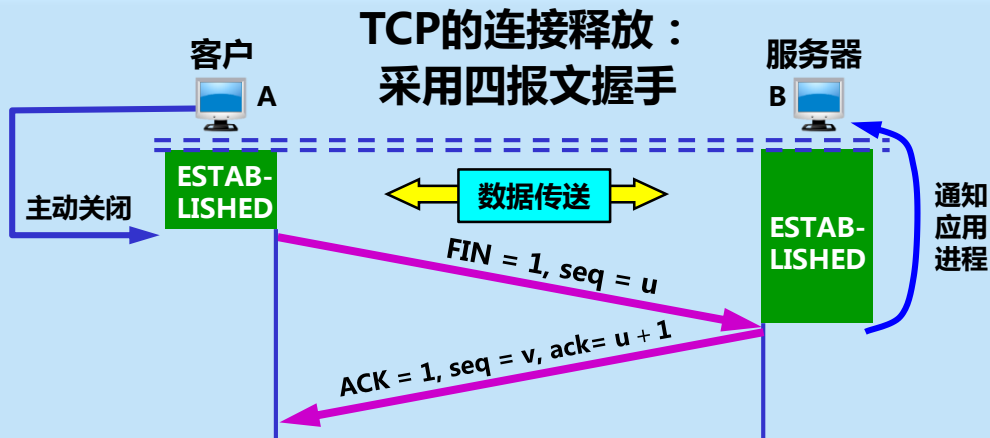


5.9.2 TCP 的连接释放

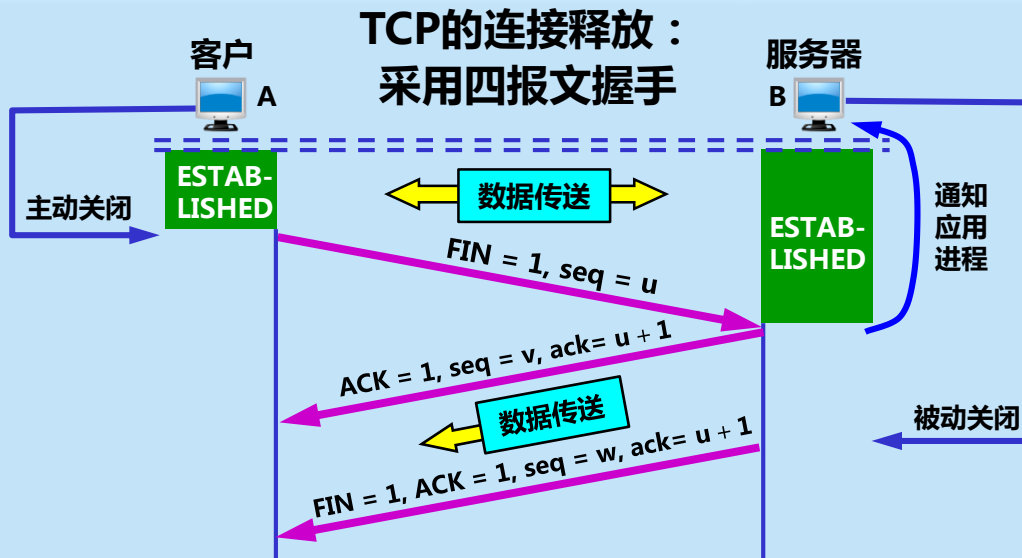
- TCP 连接释放过程比较复杂。
- 数据传输结束后，通信的双方都可释放连接。
- TCP 连接释放过程是**四报文握手**。



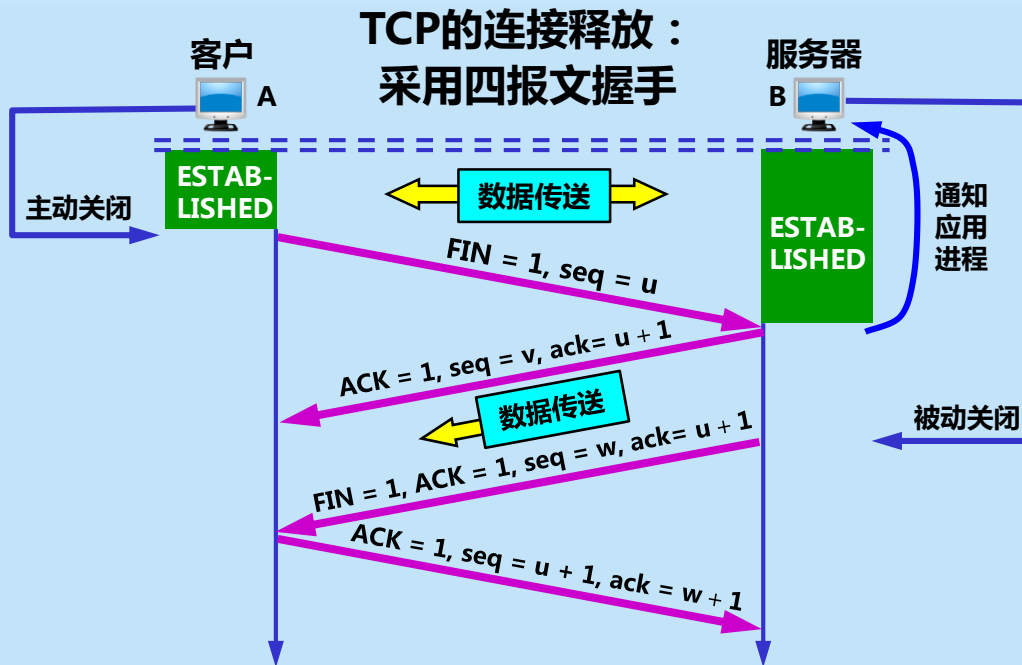
- 数据传输结束后，通信的双方都可释放连接。
- 现在 A 的应用进程先向其 TCP 发出连接释放报文段，并停止再发送数据，主动关闭 TCP 连接。
- A 把连接释放报文段首部的 $FIN = 1$ ，其序号 $seq = u$ ，等待 B 的确认。



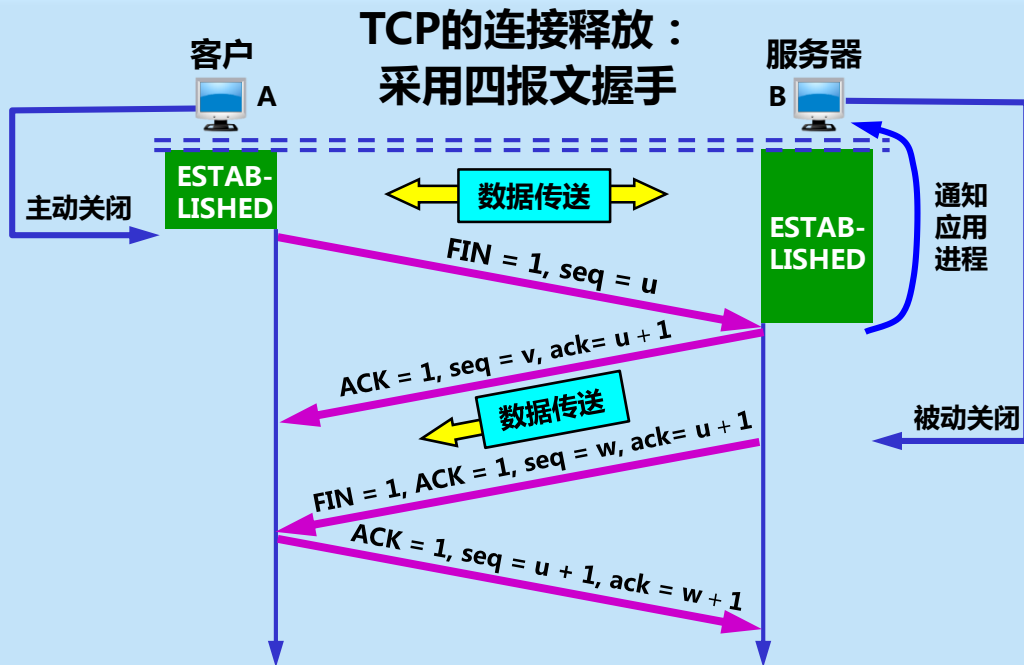
- B 发出确认，确认号 $ack = u + 1$ ，而这个报文段自己的序号 $seq = v$ 。
- TCP 服务器进程通知高层应用进程。
- 从 A 到 B 这个方向的连接就释放了，TCP 连接处于半关闭状态。B 若发送数据，A 仍要接收。



- 若 B 已经没有要向 A 发送的数据，
- 其应用进程就通知 TCP 释放连接。



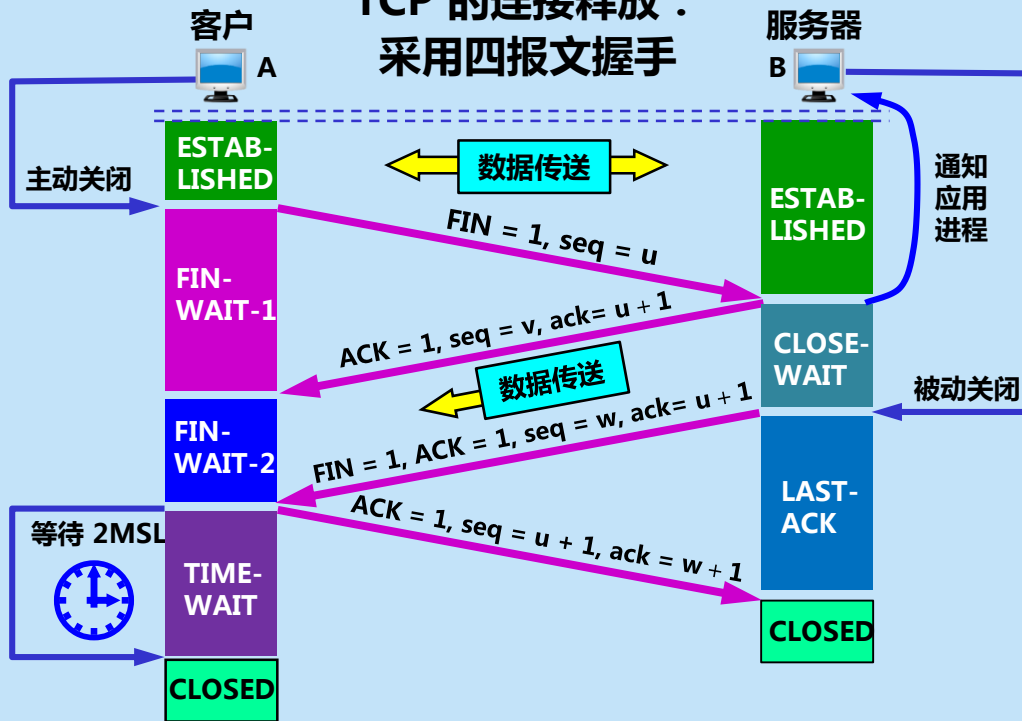
- A 收到连接释放报文段后，必须发出确认。



在确认报文段中 $ACK = 1$, 确认号 $ack = w + 1$, 自己的序号 $seq = u + 1$.



TCP 的连接释放： 采用四报文握手





A 必须等待 2MSL 的时间

- **第一**，为了保证 A 发送的最后一个 ACK 报文段能够到达 B。
- **第二**，防止 “已失效的连接请求报文段” 出现在本连接中。

MSL-最长报文段寿命，RFC793 建议为2分钟。





保活计时器

- 用来防止在TCP连接出现长时期的空闲。
- 保活计时器 通常设置为2小时。若服务器过了2小时还没有收到客户的信息，它就发送探测报文段。若发送了10个探测报文段（每一个相隔75秒）还没有响应，就假定客户出了故障，因而就终止该连接。



TCP连接建立 “三报文握手” 过程

Sniffer Portable - Local, Ethernet (Line speed at 100 Mbps) - [a2.cap: Filtered 3, 36/25000 Ethernet Frames, Filter: Matrik]				
File Monitor Capture Display Tools Database Window Help				
Default				
No	Source Addres	Dest. Addes	Summary	Len(B)
3	202.1.64.166	211.80.20.2	DNS: NAME=www.itnk.com	77
4	211.80.20.2	202.1.64.166	DNS: IP=211.80.20.200 NAME=www.itnk.com	165
5	202.1.64.166	211.80.20.200	TCP: S=1298 D=80 SYN=1 SEQ=60029	62
6	211.80.20.200	202.1.64.166	TCP: S=80 D=1298 SYN=1 ACK=1 SEQ=35601 ack=60030	62
7	202.1.64.166	211.80.20.200	TCP: S=1298 D=80 ACK=1 SEQ=60030 ack=35602	60
8	202.1.64.166	211.80.20.200	HTTP: Port=1535 GET/HTTP/1.1	568

SYN报文不能携带数据，要消耗序号



TCP连接释放“四报文握手”过程

Sniffer Portable - Local, Ethernet (Line speed at 100 Mbps) - [a2.cap: Filtered 3, 36/25000 Ethernet Frames, Filter: Matrix]			
File Monitor Capture Display Tools Database Window Help			
Default			
No	Source Addres	Dest. Addres	Summary
			
23	202.1.64.166	211.80.20.200	HTTP: Data Len=100 S=1298 D=80 seq=16651 ack=68830
24	211.80.20.200	202.1.64.166	HTTP: Data Len=1005 S=80 D=1298 seq=68831 ack=16752
25	202.1.64.166	211.80.20.200	TCP: S=1298 D=80 FIN=1 ACK=1 seq=16752 ack=69836
26	211.80.20.200	202.1.64.166	TCP: S=80 D=1298 ACK=1 seq=69836 ack=16753
27	211.80.20.200	202.1.64.166	TCP: S=80 D=1298 FIN=1 ACK=1 seq=69836 ack=16753
28	202.1.64.166	211.80.20.200	TCP: S=1298 D=80 ACK=1 seq=16753 ack=69837

不携带数据的ACK报文不消耗序号

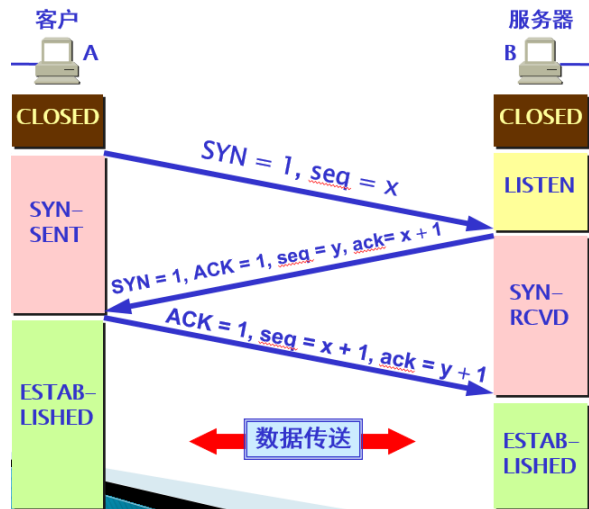


5.9.3 TCP 的有限状态机

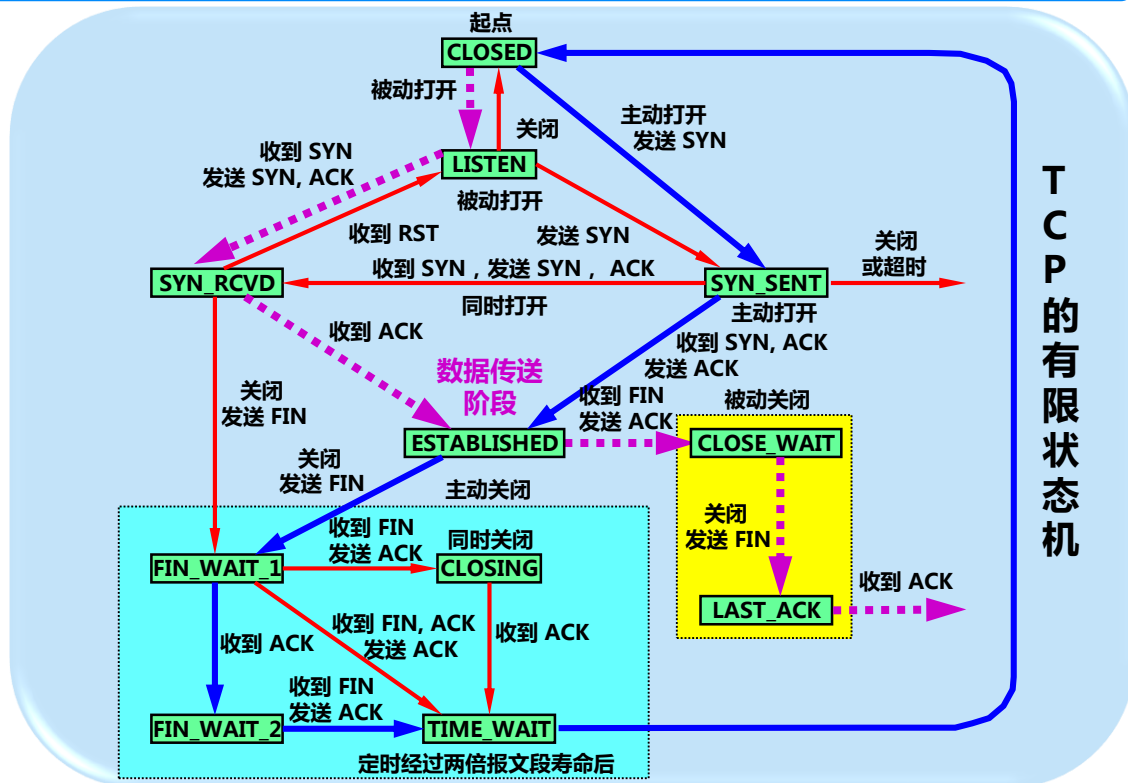
- 箭头旁边的字，表明引起这种变迁的原因，或表明发生状态变迁后又出现什么动作。
- 图中有三种不同的箭头。
 1. 粗实线箭头表示对客户进程的正常变迁。
 2. 粗虚线箭头表示对服务器进程的正常变迁。
 3. 细线箭头表示异常变迁。



5.9.3 TCP 的有限状态机



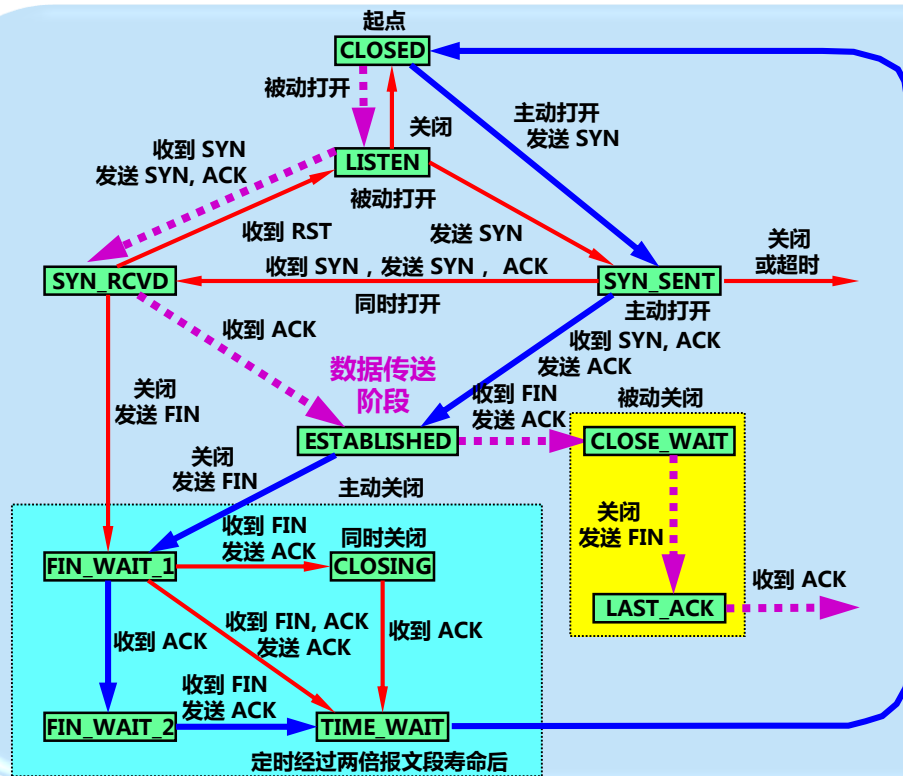
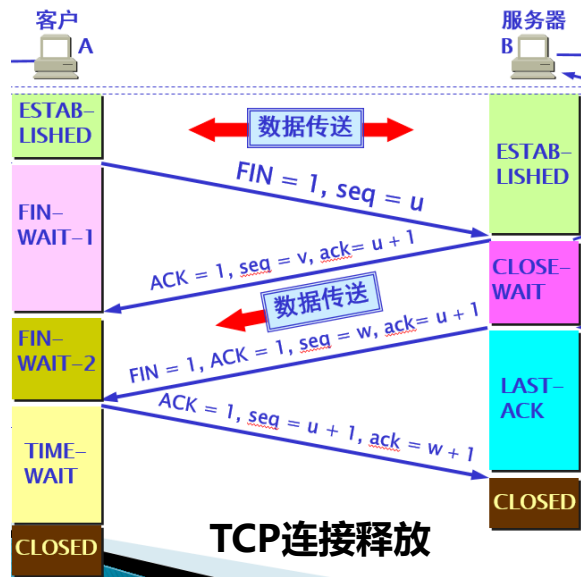
TCP连接建立



TCP 的有限状态机



5.9.3 TCP 的有限状态机



TCP 的有限状态机